

SENTENCIAS ITERATIVAS

Repetir

Las computadoras se usan a menudo para automatizar tareas repetitivas. Realizar repetidamente tareas idénticas o similares sin cometer errores es algo que las computadoras hacen bien y que los seres humanos hacemos limitadamente.



Iterar



En matemática, se refiere al proceso de iteración de una función, es decir, aplicando la función repetidamente, usando la salida de una iteración como la entrada a la siguiente.

Iterar

En programación, Iteración es la **repetición** de un **segmento de código** dentro de un programa de computadora. Puede usarse tanto como un término genérico (como sinónimo de repetición) como para describir una forma específica de repetición.



```
31
32 self.file = None
33 self.fingerprints = set()
34 self.logdupes = True
35 self.debug = debug
36 self.logger = logging.getLogger(__name__)
37 if path:
38     self.file = open(os.path.join(path, 'requests.txt'), 'a')
39     self.file.seek(0)
40     self.fingerprints.update(set(request.fingerprints))
41
42 @classmethod
43 def from_settings(cls, settings):
44     debug = settings.getbool('DEBUG', True)
45     return cls(job_dir(settings), debug)
46
47 def request_seen(self, request):
48     fp = self.request_fingerprint(request)
49     if fp in self.fingerprints:
50         return True
51     self.fingerprints.add(fp)
52     if self.file:
53         self.file.write(fp + os.linesep)
54
55 def request_fingerprint(self, request):
56     return request_fingerprint(request)
```



Tenemos una **base de datos enorme** y queremos encontrar **un dato** en especial para consultar.

Como no existe una forma mágica para encontrar el dato directamente, el programa deberá recorrer los datos uno a uno y compararlos hasta dar con el que buscamos iterando o repitiendo el mismo proceso desde el inicio comparando a ver si es el que queremos o no, así hasta que encuentre el que queremos.



Existen algoritmos que permiten ahorrarnos tiempo e iteraciones, pero en esencia sigue recorriendo uno a uno, la diferencia es que nosotros tardaríamos muchas horas, y el programa unos segundos.

¡Así que vamos a aprovecharnos de esto!



- ***WHILE***
- ***FOR***



Sentencia While

Vamos a comenzar con la sentencia iterativa más básica **While** (mientras).

Se basa en repetir un bloque de código a partir de evaluar una **condición lógica**, siempre que ésta sea **True** (al igual que la sentencia **if**).

Nosotros como programadores debemos decidir el momento en que la condición cambie a **False** para hacer que el While finalice su ejecución y así salir de la iteración, de lo contrario estaríamos frente a un **bucle infinito**.



Flujo de ejecución

Más formalmente, el flujo de ejecución de una sentencia **while** es el siguiente:

1. Evalúa la condición, devolviendo **False** o **True**.
2. Si la condición es **False**, se sale de la sentencia **While** y continúa la ejecución con la siguiente sentencia.
3. Si la condición es **True**, ejecuta cada una de las sentencias en el bloque de código y regresa al paso 1.



Bucle

Este tipo de flujo se llama **bucle** porque el **tercer paso del bucle vuelve arriba**.



Si la condición es falsa la primera vez que se pasa el bucle, las sentencias del interior del bucle no se ejecutan nunca.



Ejemplo



Analicemos qué pasó

```
>>> num = 5
>>> while num > 0:
    print(f"{num}")
    num -= 1

>>> print("Terminó el conteo!")
```

1. Declaramos una variable num y le asignamos el valor int 5.
2. Usamos la sentencia while para indicar que mientras que num sea mayor a 0 entremos al bloque de código.
3. Al evaluar num contra 0 nos indica que es True





¿Qué pasó?



4. Ingresamos al bloque de código, imprimimos num y le restamos 1 a num
5. Volvemos a repetir desde el paso 2 hasta que num deje de ser mayor a 0
6. Cuando la operación relacional de False saldremos del bucle
7. Imprimimos por pantalla Terminó el conteo!
8. Termina nuestro programa



Más ejemplos con While



```
>>> n = 0
>>> while n <= 5:
            n += 1
            print("N vale ", n)
```

¿Y un bucle infinito?

```
>>> while True:
            print("Esto es un
            bucle infinito!!!!")
```



Para escapar un bucle infinito generalmente se usa **ctrl + c**

WHILE- ELSE



Sentencia While-else

```
>>> while condicion:  
        # instrucciones de  
while
```

```
>>> else:  
        # instrucciones de  
while-else
```

```
>>> si no se abortó con break:
```

```
        # instrucciones del
```

Este **else** sirve para ejecutar un bloque de código cuando el bucle while tenga while-else una condición **False** o haya **terminado** y no haya sido forzado a salir mediante

un **break**.

Algo interesante que tiene python es que podemos encadenar un **else** (si no) al final de un bucle **while**.



¿Qué pasó?



```
>>> chance = 1
>>> while chance <= 3:
    txt = input("Escribe SI: ")
    if txt == "SI":
        print("Ok, lo conseguiste en el
intento", chance)
        break
    chance += 1
else:
    print("Has agotado tus tres intentos")
```




Sentencia While-else



1. Declaramos una variable **chance** y le asignamos el valor **int** 1.
2. Usamos la sentencia **while** para indicar que **mientras** que **chance** sea **menor** a 3 entremos al bloque de código.
3. Le pedimos al usuario que ingrese una palabra con input
4. Si la palabra es **“SI”** ingresa al condicional **if**
5. Si ingresa, imprime que lo consiguió en el intento tal y rompe el bucle con **break**
6. Si la condicional es **False** vamos a sumar uno a las chances y repetir desde el paso 2
7. Sí **chance** es mayor a 3, entramos en el **else** e imprimimos

INSTRUCCIONES

- ***break***
- ***continue***
- ***pass***



Bucles en Python

Usar bucles en Python nos permite automatizar y repetir tareas de manera eficiente.

Sin embargo, a veces, es posible que un **factor externo influya en la forma en que se ejecuta su programa.**



Instrucciones

Cuando esto sucede, es posible que prefiramos que nuestro programa cierre un bucle por completo, omita parte de un bucle antes de continuar o ignore ese factor externo.

Para hacer estas acciones python nos brinda las **instrucciones break, continue y pass.**

BREAK



Break

Comencemos por uno de los más sencillos y más utilizados: El **break**.

En Python, la instrucción **break** le proporciona la oportunidad de **cerrar un bucle** cuando se **activa** una **condición externa**. Debe poner la instrucción **break** dentro del bloque de código bajo la instrucción de su bucle, generalmente **después** de una sentencia **if** condicional.



Ejemplo



```
>>> n = 5:
>>> while n < 10:
            n -= 1
            if n == 2:
                print("ahora que n
vale 2 salimos")
                break
            print("n vale ", n)
```

Analicemos qué pasó



¿Qué pasó?



1. Declaramos una variable **n** y le asignamos el valor **int** 5.
2. Usamos la sentencia **while** para indicar que **mientras** que **n** sea **menor** a 10 entremos al bloque de código.
3. Le restamos 1 a n
4. Usamos una sentencia **if** condicional para igualar n a 2
5. Si da **True** imprime ahora que n vale 2 salimos y con **break** sale del bucle, ya no se ejecuta el resto
6. Si da **False** imprime que n vale n y vuelve al paso 2.

CONTINUE



Continue

La instrucción **continue** da la opción de **omitir** la parte de un bucle en la que se **activa** una condición externa, pero continuar para completar el resto del bucle. Es decir, la **iteración actual del bucle se interrumpirá, pero el programa volverá a la parte superior del bucle.**

Debe poner la instrucción **continue** dentro del bloque de código bajo la instrucción de su bucle, generalmente **después** de una sentencia **if** condicional.



Continue



Veamos un ejemplo:

```
>>> n = 0:
```

```
>>> while n < 10:
```

```
    n += 1
```

```
    if n == 2:
```

```
        print("Continuamos con la siguiente iteración")
```

```
            continue
```

```
        print("n vale ", n)
```

Analicemos qué pasó



¿Qué pasó?

1. Declaramos una variable **n** y le asignamos el valor **int** 0.
2. Usamos la sentencia **while** para indicar que **mientras** que **n** sea **menor** a 10 entremos al bloque de código.
3. Le sumamos 1 a n
4. Usamos una sentencia **if** condicional para igualar n a 2
5. Si da **True** imprime **Continuamos con la siguiente iteración** y con **continue** rompe la iteración, pero no de todo el bucle, sólo de esta iteración
6. Si da **False** imprime que n vale n y vuelve al paso 2.

PASS



Pass

Cuando se activa una condición externa, la instrucción **pass** permite manejar la condición **sin que el bucle se vea afectado** de ninguna manera; todo el código continuará leyéndose a menos que se produzca la instrucción break u otra instrucción.

Debe poner la instrucción **pass** dentro del bloque de código bajo la instrucción de su bucle, generalmente **después** de una sentencia **if** condicional.



Ejemplo

```
>>> n = 0:
>>> while n < 10:
            n += 1
            if n == 2:
                pass
            print("n vale ", n)
```




¿Qué pasó?



1. Declaramos una variable **n** y le asignamos el valor **int** 0.
2. Usamos la sentencia **while** para indicar que **mientras** que **n** sea **menor** a 10 entremos al bloque de código.
3. Le sumamos 1 a n
4. Usamos una sentencia **if** condicional para igual n a 2
5. Si da **True** con la instrucción pass le indica al programa que continúe ejecutando el bucle e **ignore** el hecho de que la variable n se evalúa como equivalente a 2 durante una de sus iteraciones
6. Si da **False** imprime que n vale n, le resta 1 y vuelve al paso 2.

FOR

(repetir, pero conociendo la cantidad de repeticiones)



Sentencia For

Ahora seguiremos con la sentencia iterativa que podríamos decir es la más usada **For** (para).

Se utiliza para recorrer los elementos de un objeto iterable (lista, tupla...) y ejecutar un bloque de código, o sea, tiene un número predeterminado de veces que itera.



Sentencia For

En cada paso de la iteración se tiene en cuenta a un único elemento del objeto iterable, sobre el cuál se pueden aplicar una serie de operaciones.

```
>>> lista = [1,2,3,4,5]
```

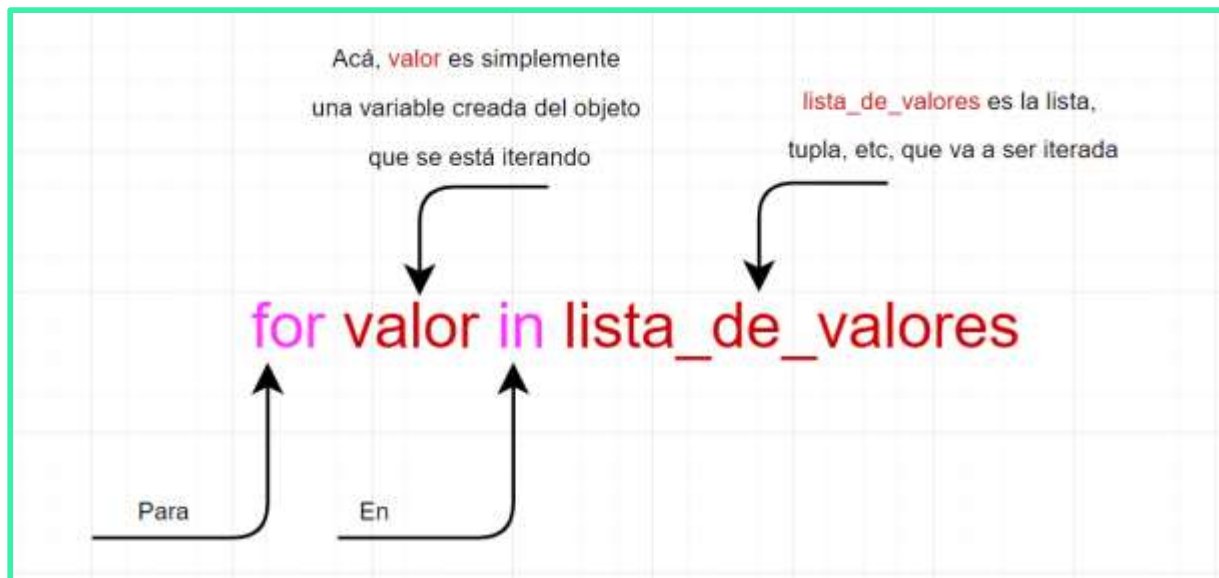
```
>>> for valor in lista:
```

```
    print("Soy un item de la lista y valgo “,
```

```
valor)
```



Ejemplo gráfico



`valor` simplemente es una copia local, no afecta fuera del bucle a menos que se devuelva el valor



Ejemplo



```
>>> lista = [0,1,2,3,4,5,6,7,8,9,10]
```

```
>>> for num in lista:
```

```
    print("Soy un valor de la lista y valgo ", num)
```

```
    num *= 5
```

```
    print("Soy un valor de la lista y ahora valgo
```

```
    ", num)
```



Modificando la lista

```
>>> indice = 0
>>> numeros = [0,1,2,3,4,5,6,7,8,9,10]
>>> for numero in numeros:
            numeros[indice] *= 5
            indice += 1
            # tambien se puede con indice =
numero
>>> print(numeros)
```

ENUMERATE

1 =

2 =

3 =

Enumerate

La función incorporada `enumerate(lista/tupla_de_valores)` toma como argumento un objeto iterable y retorna otro cuyos elementos son tuplas de dos objetos:

1. El primero de los dos indica la posición de un elemento perteneciente a el objeto iterable, es decir, el índice.
2. El segundo, el elemento mismo.

☞ Esto se conoce como lectura secuencial de clave y valor, lo vamos a usar bastante en el futuro!

For + Enumerate



Si quisiéramos modificar la lista:

```
lista = [66,-3,35,96]
#---Pos ,   elem
for indice, numero in enumerate(lista):

    print(f"INDICE: {indice}")
    print(f"NUMERO: {numero}\n\n")

    print(f"-----> {lista[indice]}\n\n-----")
```

Sentencia For

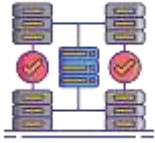


Si quisiéramos recorrer un string:

```
>>> texto = "Hola Mundo, estoy usando for en Python"
>>> for letra in texto:
        print(letra)

>>> texto2 = ""
>>> for letra in texto:
        texto2 = letra * 2
>>> print(texto2)
```

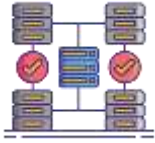
RANGE



¿Qué es?

Como vimos, el **for** en python **necesita** una colección de datos para poder utilizarlo, en otros lenguajes necesitamos solamente un número para indicar las iteraciones a cumplir.

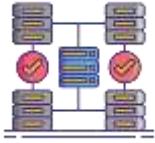
Para simular estos casos Python nos provee de una función denominada **range()** (rango) el cual **representa una colección de números inmutables.**



Constructores para crear objetos Range



1. range(**fin**): Crea una secuencia numérica que va desde 0 hasta fin - 1.
 - a. for numero in **range**(10)
2. range(inicio, fin): Crea una secuencia numérica que va desde inicio hasta fin - 1.
 - a. for numero in range(5, 10)
3. range(inicio, fin, [paso]): Crea una secuencia numérica que va desde inicio hasta fin - 1. Si además se indica el parámetro paso, la secuencia genera los números de paso en paso.
 - a. for numero in range(0, 20, 2)



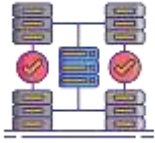
Ventajas



```
>>> range(0,10)
```

Parece ser exactamente una lista que va de 0 a 10, pero range interpreta el inicio y fin en tiempo de ejecución y eso le da ventaja contra la lista.

Si tuviéramos una lista de 0 a 10000 estaría ocupando muchísimo espacio en memoria.



Ventajas



```
>>> range(0,10000)
```

Ahora, si hiciéramos `range(0,10000)`, `range` interpreta esto en tiempo de ejecución, es decir cuando se ejecuta el 0 se crea el 0 cuando se ejecuta el 1 se crea el 1 y se elimina el 0 y así continuamente, y esto no ocupa memoria

For-else



Igual que en la sentencia while podemos usar un **else** al final de la iteración

```
>>> for numero in range(10):  
        print("Numero vale ",numero)  
    else:  
        print("Se terminó de iterar y numero vale: ",  
numero)
```

For-break-continue-pass



```
>>> for numero in range(10):
        if numero == 2:
            continue
        elif numero == 8:
            break
        print("Numero vale ",numero)
    else:
        print("Se terminó de iterar y numero vale:
", numero)
```

Igual que en la sentencia **while** podemos usar también las instrucciones **break continue** y **pass**

PUESTA EN COMÚN

¡Analicemos por última vez!

```
n = 0
while n < 10:
    if (n%2) == 0:
        print(n, 'es un número
par')
    else:
        print(n, 'es un número
impar')
```

¡Ahora podemos entender que hace este programa que vimos hace unas
clases!